



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش پنجم

طراحی ضرب‌کننده (روش Booth)

خلاصه

در این آزمایش یک واحد ضرب‌کننده‌ی علامت‌دار با روش بوث (Booth) طراحی شده است. طبق خواسته‌ی صورت آزمایش، مسیر داده (datapath) و واحد کنترل (control unit) به صورت جداگانه طراحی و سپس در یک مازول سطح بالا به یکدیگر متصل شده‌اند.

نکته‌ی کلیدی صورت آزمایش این است که واحد شیفت‌دهنده باید بتواند بیش از یک بیت در هر پالس ساعت شیفت بدهد تا نسبت به ضرب عادی (Shift & Add) تسریع داشته باشد. برای برآورده کردن این شرط، از بوث پایه-۴ (radix-4 یا modified Booth) استفاده شده است که در هر تکرار دو بیت از مضروب‌فیه را مصرف می‌کند و مجموعه‌ی $\{A, Q, Q_{-1}\}$ را دو بیت به راست شیفت می‌دهد. نتیجه این است که برای عملوند N بیتی به جای N تکرار، تنها $N/2$ تکرار لازم است.

صحت‌سنجی با Icarus Verilog انجام شده و ضرب‌کننده روی تمام فضای ورودی علامت‌دار ۸ در ۸ بیت (هر $256 \times 256 = 65536$ جفت) با عملکرد ضرب علامت‌دار خود Verilog مقایسه شده است.

چرا پایه-۴؟ (شرط تسریع)

در روش Shift & Add معمولی و همچنین بوث پایه-۲، در هر پالس ساعت تنها یک بیت از مضروب‌فیه بررسی و یک بیت شیفت انجام می‌شود؛ پس ضرب دو عدد N بیتی N کلاک طول می‌کشد.

در بوث پایه-۴، در هر تکرار یک پنجره‌ی سه‌بیتی از $\{Q_{-1}, Q_0, Q_1\}$ بررسی می‌شود و براساس آن یکی از پنج مضرب $\{0, +M, -M, +2M, -2M\}$ به انباشتگر اضافه می‌شود، سپس شیفت دو بیتی انجام می‌گیرد. جدول بازکدگذاری (recoding) به این صورت است:

تفسیر	عملیات	Q_{-1}	Q_0	Q_1
دنباله‌ی صفر	$A += 0$	۰	۰	۰
پایان دنباله‌ی یک	$A += M$	۱	۰	۰
یک تکی	$A += M$	۰	۱	۰
پایان دنباله‌ی یک	$A += 2M$	۱	۱	۰
آغاز دنباله‌ی یک	$A -= 2M$	۰	۰	۱
صفر تکی	$A -= M$	۱	۰	۱
آغاز دنباله‌ی یک	$A -= M$	۰	۱	۱
دنباله‌ی یک	$A += 0$	۱	۱	۱

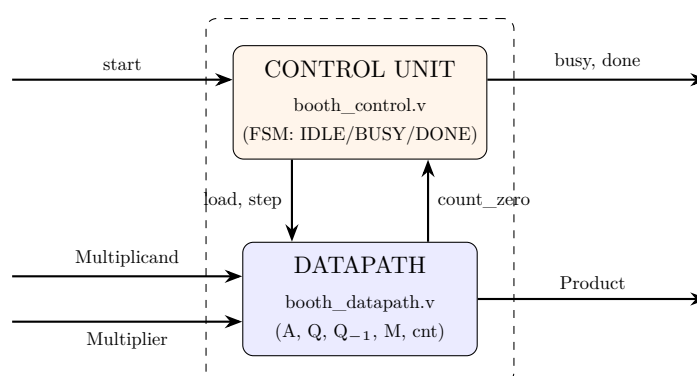
مقایسه‌ی تعداد کلاک‌ها که در تست‌بنچ هم اندازه‌گیری شده است:

N	پایه ۲ / Shift & Add	پایه ۴ (این طراحی) تسریع
۴	۵ کلاک	۳ کلاک $1/7 \times$
۸	۹ کلاک	۵ کلاک $1/8 \times$
۱۶	۱۷ کلاک	۹ کلاک $1/9 \times$

اعداد ستون سوم مستقیماً از خودِ مدار خوانده شده‌اند، نه اینکه فرض شده باشند؛ تست‌بنچ لبه‌های کلاک را تا فعال شدن done می‌شمارد.

معماری کلی

طراحی از سه ماژول تشکیل شده است:



ماژول	مسئولیت
booth_datapath.v	همه‌ی رجیسترها و محاسبات: انباشتگر A ، مضروب Q ، بیت Q_{-1} ، مضروب M ، شمارنده، بازکدگذاری بوث و شیفت دوبیتی. هیچ تصمیم‌تربیتی نمی‌گیرد.
booth_control.v	ماشین حالت Moore با سه حالت. فقط تعیین می‌کند چه زمانی عملوندها بارگذاری و چه زمانی یک تکرار اجرا شود. هیچ داده‌ای نگه نمی‌دارد.
booth_multiplier.v	اتصال این دو به یکدیگر؛ خودش منطقی ندارد.

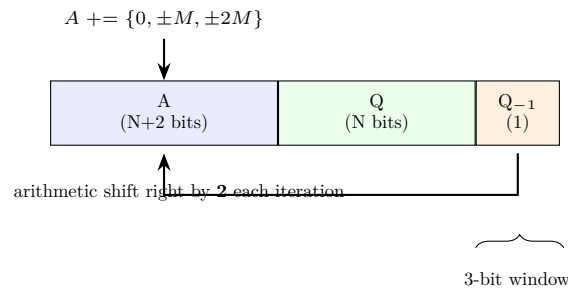
مشخصات واسط

سیگنال	جهت	پهنای	توضیح
Clk	ورودی	۱	کلاک؛ لبه‌ی بالارونده
RstN	ورودی	۱	ریست ناهم‌گام فعال‌پایین
start	ورودی	۱	درخواست شروع یک ضرب جدید
Multiplicand	ورودی	N	مضروب (علامت‌دار، مکمل ۲)
Multiplier	ورودی	N	مضروب‌فیه (علامت‌دار، مکمل ۲)
Product	خروجی	$2N$	حاصل ضرب علامت‌دار
busy	خروجی	۱	یعنی ضرب در جریان است
done	خروجی	۱	یعنی نتیجه آماده است

سیگنال done از نوع سطح است و تا رسیدن start بعدی بالا می‌ماند، بنابراین یک مصرف‌کننده‌ی کند هم نتیجه را از دست نمی‌دهد.

مسیر داده

ساختار رجیسترها همان چیدمان کلاسیک بوث است:



نکته‌ای که در پیاده‌سازی اهمیت زیادی دارد پهنای انباشتگر است. در بسیاری از توضیحات درسی، A را $N + 1$ بیت می‌گیرند؛ اما در پایه-۴ این کافی نیست. اگر مضروب برابر منفی‌ترین مقدار ممکن یعنی $M = -2^{N-1}$ باشد، آنگاه:

$$2M = -2^N \quad \text{و} \quad -2M = +2^N$$

مقدار 2^N در $N + 1$ بیت علامت‌دار جا نمی‌شود (بازه‌ی آن $[-2^N, 2^N - 1]$ است). بنابراین در این طراحی A و مقدار افزودنی هر دو $N + 2$ بیت گرفته شده‌اند تا هر پنج مضرب دقیقاً نمایش‌پذیر باشند و منفی‌ترین عملوند نیازی به حالت خاص نداشته باشد.

این دقیقاً همان اشکالی بود که در اولین اجرای تست ظاهر شد: تمام ۸۶ خطای اولیه مربوط به حالت Multiplicand = ۱۲۸- بود و پس از افزایش پهنای انباشتگر به $N + 2$ برطرف شد.

```

1 // Radix-4 Booth multiplier - DATAPATH.
2 //
3 // This module holds all the storage and arithmetic; it contains no
4 // sequencing decisions of its own. Every state change is requested by the
5 // control unit through the `load` / `step` strobes.
6 //
7 // Algorithm (modified / radix-4 Booth)
8 // -----
9 // The accumulator register triple {A, Q, Q_1} is the classic Booth layout:
10 //
11 //   A (N+2 bits, signed)  Q (N bits)  Q_1 (1 bit)
12 //   \__ running partial product __/   \_ the bit shifted out last time
13 //
14 // Each iteration inspects the low three bits {Q[1], Q[0], Q_1} and adds one
15 // of five multiples of M to A, then shifts the whole triple RIGHT BY TWO
16 // arithmetically:
17 //
18 //   Q[1] Q[0] Q_1 | operation
19 //   -----+-----
20 //   0   0   0 | A += 0
21 //   0   0   1 | A += M
22 //   0   1   0 | A += M
23 //   0   1   1 | A += 2M
24 //   1   0   0 | A -= 2M
25 //   1   0   1 | A -= M
26 //   1   1   0 | A -= M
27 //   1   1   1 | A += 0
28 //
29 // Because two multiplier bits are retired per step, an N-bit operand needs
30 // only N/2 iterations instead of N - this is exactly the "shift more than
31 // one bit per clock" speed-up the lab asks for.
32 //
33 // Accumulator width: A is TWO bits wider than N. One extra bit is the usual
34 // Booth sign/guard bit, and a second is required because the -2M multiple
35 // must be representable: with M = -2^(N-1) (the most negative operand),
36 // 2M = -2^N and -2M = +2^N, which does NOT fit in N+1 signed bits. Sizing A

```

```

37 // and the addend to N+2 bits makes every one of the five multiples exact, so
38 // the most-negative multiplicand needs no special-casing.
39 module booth_datapath #(
40     parameter integer N = 8                // operand width; must be even
41 ) (
42     input wire          Clk,
43     input wire          RstN,              // async, active-low
44
45     // --- control inputs -----
46     input wire          load,              // capture operands, clear acc
47     input wire          step,              // perform one radix-4 iteration
48
49     // --- operands -----
50     input wire signed [N-1:0] Multiplicand,
51     input wire signed [N-1:0] Multiplier,
52
53     // --- status back to the control unit -----
54     output wire [$clog2(N/2):0] count,      // iterations still to run
55     output wire          count_zero,        // 1 when no iteration is left
56
57     // --- result -----
58     output wire signed [2*N-1:0] Product
59 );
60 localparam integer ITER = N / 2;           // radix-4 => N/2 steps
61 localparam integer CNT_W = $clog2(N/2) + 1;
62
63 // --- registers -----
64 reg signed [N+1:0] A;                      // partial product, two guard bits (see above)
65 reg [N-1:0] Q;                             // multiplier, consumed 2 bits per step
66 reg Q_1;                                   // the bit that fell off the bottom
67 reg signed [N-1:0] M;                      // multiplicand, held constant
68 reg [CNT_W-1:0] cnt;                      // remaining iterations
69
70 assign count = cnt;
71 assign count_zero = (cnt == 0);
72
73 // The product is the low 2N bits of {A, Q} after the last shift.
74 assign Product = {A[N-1:0], Q};
75
76 // ---- Booth recoding of the current 3-bit window -----
77 wire [2:0] booth_window = {Q[1], Q[0], Q_1};
78
79 // Sign-extended multiples of M, in the A register's width (N+2 bits).
80 wire signed [N+1:0] M_ext = {{2{M[N-1]}}, M}; // 1*M
81 wire signed [N+1:0] M2_ext = {M[N-1], M, 1'b0}; // 2*M
82
83 reg signed [N+1:0] addend;
84 always @(*) begin
85     case (booth_window)
86         3'b000, 3'b111: addend = {(N+2){1'b0}}; // 0
87         3'b001, 3'b010: addend = M_ext;         // +M
88         3'b011:         addend = M2_ext;         // +2M
89         3'b100:         addend = -M2_ext;         // -2M
90         3'b101, 3'b110: addend = -M_ext;         // -M
91         default:        addend = {(N+2){1'b0}};
92     endcase
93 end
94
95 wire signed [N+1:0] A_sum = A + addend;
96
97 // ---- arithmetic shift right by TWO of the triple {A, Q, Q_1} -----
98 // The two bits leaving Q's bottom: Q[1] is discarded, Q[0] becomes the
99 // new Q_1 for the next window.
100 wire signed [N+1:0] A_next = {A_sum[N+1], A_sum[N+1], A_sum[N+1:2]};
101 wire [N-1:0] Q_next = {A_sum[1:0], Q[N-1:2]};
102 wire Q_1_next = Q[1];
103
104 // ---- sequential update -----
105 always @(posedge Clk or negedge RstN) begin
106     if (!RstN) begin
107         A <= {(N+2){1'b0}};
108         Q <= {N{1'b0}};
109         Q_1 <= 1'b0;
110         M <= {N{1'b0}};
111         cnt <= {CNT_W{1'b0}};
112     end else if (load) begin
113         A <= {(N+2){1'b0}};
114         Q <= Multiplier;
115         Q_1 <= 1'b0;
116         M <= Multiplicand;
117         cnt <= ITER[CNT_W-1:0];
118     end else if (step && cnt != 0) begin
119         A <= A_next;
120         Q <= Q_next;
121         Q_1 <= Q_1_next;
122         cnt <= cnt - 1'b1;
123     end
124 end

```

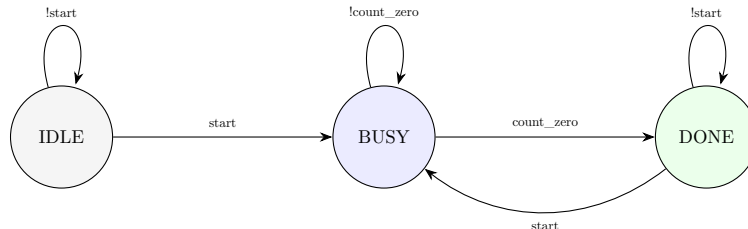
```

124 end
125 endmodule

```

واحد کنترل

واحد کنترل یک ماشین حالت Moore با سه حالت است:



حالت	load	step	توضیح
IDLE	start	۰	منتظر درخواست؛ با start عملوندها بارگذاری می‌شوند
BUSY	۰	۱	در هر کلاک یک تکرار پایه-۴ اجرا می‌شود
DONE	start	۰	نتیجه معتبر است و تا start بعدی می‌ماند

توجه کنید که واحد کنترل خودش شمارنده ندارد؛ شمارنده در مسیر داده است و کنترل فقط پرچم وضعیت count_zero را می‌بیند. این تفکیک باعث می‌شود تغییر عمق تکرار (مثلاً با تغییر N) هیچ اثری روی واحد کنترل نداشته باشد.

```

1 // Radix-4 Booth multiplier - CONTROL UNIT.
2 //
3 // A three-state Moore FSM. It owns no data: it only decides *when* the
4 // datapath loads operands and *when* it performs an iteration, and it
5 // watches the datapath's `count_zero` status flag to know when to stop.
6 //
7 // IDLE --start--> BUSY --count_zero--> DONE --start--> BUSY
8 //
9 // |_____ (else) _____|
10 //
11 // `busy` is high while a multiplication is in flight, `done` is a level
12 // that stays high until the next `start`, so a slow consumer cannot miss
13 // the result.
14 module booth_control #(
15     parameter integer N = 8
16 ) (
17     input wire Clk,
18     input wire RstN,           // async, active-low
19
20     input wire start,          // request a new multiplication
21     input wire count_zero,     // status from the datapath
22
23     output reg load,           // strobe: capture operands
24     output reg step,           // strobe: run one radix-4 iteration
25     output wire busy,
26     output wire done
27 );
28 localparam [1:0] S_IDLE = 2'd0,
29                 S_BUSY = 2'd1,
30                 S_DONE = 2'd2;
31
32 reg [1:0] state, next_state;
33
34 // ---- state register -----
35 always @(posedge Clk or negedge RstN) begin
36     if (!RstN) state <= S_IDLE;
37     else state <= next_state;
38 end
39

```

```

40 // ---- next-state logic -----
41 always @(*) begin
42     case (state)
43         S_IDLE: next_state = start      ? S_BUSY : S_IDLE;
44         // Leave BUSY only once the datapath reports no iteration left.
45         S_BUSY: next_state = count_zero ? S_DONE : S_BUSY;
46         S_DONE: next_state = start      ? S_BUSY : S_DONE;
47         default: next_state = S_IDLE;
48     endcase
49 end
50
51 // ---- output logic -----
52 // `load` fires on the transition into BUSY, so operands are captured on
53 // the same edge the FSM enters BUSY; `step` runs throughout BUSY.
54 always @(*) begin
55     load = 1'b0;
56     step = 1'b0;
57     case (state)
58         S_IDLE: load = start;
59         S_BUSY: step = 1'b1;
60         S_DONE: load = start;
61         default: ;
62     endcase
63 end
64
65 assign busy = (state == S_BUSY);
66 assign done = (state == S_DONE);
67 endmodule

```

اتصال دو واحد

ماژول سطح بالا فقط این دو را به هم وصل می‌کند و منطق مستقلی ندارد:

```

1 // Radix-4 Booth multiplier - TOP LEVEL.
2 //
3 // The lab asks for the datapath and the control unit to be designed
4 // separately and then joined; this module is exactly that join and contains
5 // no logic of its own beyond the wiring.
6 //
7 //          start
8 //          done      CONTROL
9 //          busy      (FSM)
10 //          load      step
11 //
12 //
13 //
14 // Multiplicand, Multiplier  DATAPATH  Product
15 //
16 //          count_zero  (back to control)
17 //
18 // Timing: after `start`, the result is valid when `done` rises, which takes
19 // N/2 + 1 clocks (N/2 radix-4 iterations plus the →IDLEBUSY transition).
20 module booth_multiplier #(
21     parameter integer N = 8          // operand width; must be even
22 ) (
23     input wire          Clk,
24     input wire          RstN,        // async, active-low
25     input wire          start,
26     input wire signed [N-1:0] Multiplicand,
27     input wire signed [N-1:0] Multiplier,
28     output wire signed [2*N-1:0] Product,
29     output wire          busy,
30     output wire          done
31 );
32 // control <-> datapath handshake
33 wire load, step, count_zero;
34
35 booth_control #(N(N)) u_control (
36     .Clk          (Clk),
37     .RstN         (RstN),
38     .start        (start),
39     .count_zero   (count_zero),
40     .load         (load),
41     .step         (step),
42     .busy         (busy),
43     .done         (done)
44 );
45
46 booth_datapath #(N(N)) u_datapath (
47     .Clk          (Clk),
48     .RstN         (RstN),
49     .load         (load),

```

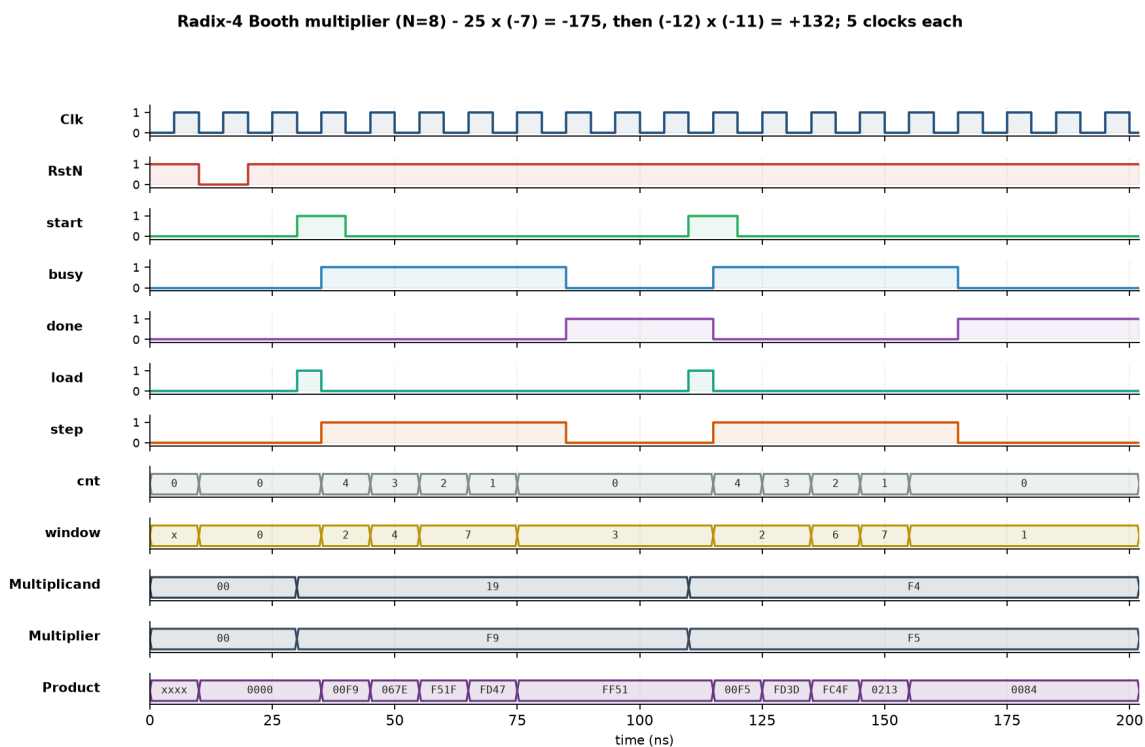
```

50     .step      (step),
51     .Multiplicand (Multiplicand),
52     .Multiplier  (Multiplier),
53     .count      (), // exposed for debug/waveforms only
54     .count_zero  (count_zero),
55     .Product     (Product)
56 );
57 endmodule

```

شکل موج

شکل موج دو ضرب پشت‌سرهم را نشان می‌دهد: ابتدا $(-7) \times 25 = -175$ و سپس $(-11) \times (-12) = +132$ ؛ یعنی هر دو حالت افزودن و کاستن در بازکدگذاری بوث دیده می‌شود.



شکل ۱ - ضرب‌کننده بوث پایه-۴ با $N = 8$

نکات قابل مشاهده در شکل: شمارنده‌ی cnt دنباله‌ی $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ را طی می‌کند، یعنی دقیقاً چهار تکرار برای عملوند ۸ بیتی و نه هشت تکرار. سیگنال window همان پنجره‌ی سه‌بیتی است که در هر تکرار عوض می‌شود. مقدار Product در پایان ضرب اول برابر FF51 است که در مکمل ۲ همان -175 می‌شود، و در پایان ضرب دوم 0084 یعنی $+132$. همچنین دیده می‌شود که load تنها یک پالس تک‌کلاکی است در حالی که step در سراسر حالت BUSY فعال می‌ماند.

تست بنچ اصلی

تست بنچ اصلی حاصل ضرب را با عملگر ضرب علامت‌دار خود Verilog مقایسه می‌کند و کل فضای ورودی ۸ در ۸ بیت را به صورت exhaustive می‌پیماید. علاوه بر آن، حالت‌های مرزی کلاسیک به صورت جداگانه هم آزموده شده‌اند: صفرها، ± 1 ، بزرگ‌ترین مثبت‌ها، منفی‌ترین مقدار -128 (از جمله -128×-128)، الگوهای متناوب 0xAA/0x55، شروع‌های پشت‌سرهم بدون ریست، و ریست در میانه‌ی یک ضرب.

```

1 `timescale 1ns/1ps
2
3 // Self-checking testbench for the radix-4 Booth multiplier.
4 //
5 // The reference is Verilog's own signed `*` operator, so the test proves the
6 // Booth recoding + arithmetic shift produce exactly the textbook product
7 // over the ENTIRE signed input space (all 256x256 = 65536 pairs).
8 module tb_booth_multiplier;
9     localparam integer N = 8;
10
11     reg          Clk, RstN, start;
12     reg signed [N-1:0] Multiplicand, Multiplier;
13     wire signed [2*N-1:0] Product;
14     wire          busy, done;
15
16     booth_multiplier #(N) dut (
17         .Clk(Clk), .RstN(RstN), .start(start),
18         .Multiplicand(Multiplicand), .Multiplier(Multiplier),
19         .Product(Product), .busy(busy), .done(done)
20     );
21
22     integer pass, fail;
23     integer ai, bi;
24     integer cycles, max_cycles, min_cycles;
25     reg signed [2*N-1:0] expected;
26     // Signed holders: a part-select like ai[N-1:0] is unsigned in Verilog,
27     // so the loop values must pass through signed regs before being used
28     // either as stimulus or inside the `a * b` reference below.
29     reg signed [N-1:0] sa, sb;
30
31     initial Clk = 1'b0;
32     always #5 Clk = ~Clk;
33
34     // Drive one multiplication and wait for `done`, counting the clocks it
35     // actually took so the speed-up claim can be measured, not assumed.
36     task automatic multiply;
37         input signed [N-1:0] a;
38         input signed [N-1:0] b;
39         begin
40             @(negedge Clk);
41             Multiplicand = a;
42             Multiplier = b;
43             start = 1'b1;
44             @(negedge Clk);
45             start = 1'b0;
46
47             // Latency = clock edges from the one that sampled `start`
48             // until `done` is observed high.
49             cycles = 0;
50             while (!done) begin
51                 @(posedge Clk); #1;
52                 cycles = cycles + 1;
53                 if (cycles > 100) begin
54                     $display("FAIL: timeout waiting for done (a=%0d b=%0d)", a, b);
55                     $fatal(1);
56                 end
57             end
58             #1;
59         end
60     endtask
61
62     task automatic check;
63         input signed [N-1:0] a;
64         input signed [N-1:0] b;
65         begin
66             multiply(a, b);
67             expected = a * b;
68             if (Product === expected) begin
69                 pass = pass + 1;
70             end else begin
71                 $display("FAIL a=%0d b=%0d got=%0d (%h) exp=%0d (%h)",
72                     a, b, Product, Product, expected, expected);
73                 fail = fail + 1;
74             end
75             if (cycles > max_cycles) max_cycles = cycles;
76             if (cycles < min_cycles) min_cycles = cycles;
77         end
78     endtask
79
80     task automatic do_reset;
81         begin
82             @(negedge Clk);
83             start = 1'b0; Multiplicand = 0; Multiplier = 0;
84             RstN = 1'b0;
85             @(negedge Clk);
86             RstN = 1'b1;

```



```

87     #1;
88     if (busy !== 1'b0 || done !== 1'b0) begin
89         $display("FAIL: flags not clear after reset (busy=%b done=%b)",
90             busy, done);
91         fail = fail + 1;
92     end else pass = pass + 1;
93 end
94 endtask
95
96 initial begin
97     pass = 0; fail = 0;
98     max_cycles = 0; min_cycles = 1000;
99     RstN = 1'b1; start = 0; Multiplicand = 0; Multiplier = 0;
100
101     do_reset;
102
103     // --- directed corner cases (the ones Booth classically gets wrong) --
104     check( 0, 0);
105     check( 0, 57);
106     check( 57, 0);
107     check( 1, 1);
108     check( 1, -1);
109     check(-1, 1);
110     check(-1, -1);
111     check(127, 127); // largest positives
112     check(127, -128);
113     check(-128, 127);
114     check(-128, -128); // most negative squared: needs the guard bit
115     check(-128, 1);
116     check( 1, -128);
117     check(-128, -1);
118     check(-1, -128);
119     check( 85, -86); // alternating bit patterns 0x55 / 0xAA
120     check(-86, 85);
121     check( 85, 85);
122     check(-86, -86);
123
124     // --- back-to-back starts without an intervening reset -----
125     check( 12, 10);
126     check(-12, 10);
127     check( 12, -10);
128     check(-12, -10);
129
130     // --- reset in the middle of a multiplication -----
131     @(negedge Clk);
132     Multiplicand = 100; Multiplier = 100; start = 1'b1;
133     @(negedge Clk); start = 1'b0;
134     @(posedge Clk); // let it get part-way through
135     @(posedge Clk);
136     do_reset;
137     check(6, 7); // must still work correctly afterwards
138
139     // --- EXHAUSTIVE: every signed 8x8 pair -----
140     for (ai = -128; ai <= 127; ai = ai + 1)
141         for (bi = -128; bi <= 127; bi = bi + 1) begin
142             sa = ai[N-1:0];
143             sb = bi[N-1:0];
144             check(sa, sb);
145         end
146
147     $display("cycles per multiply: min=%0d max=%0d (N=%0d, radix-4 => %0d iterations)",
148         min_cycles, max_cycles, N, N/2);
149     $display("tb_booth_multiplier: Passed: %0d, Failed: %0d", pass, fail);
150     if (fail != 0) $fatal(1);
151     $finish;
152 end
153 endmodule

```

تست‌بنچ پارامتری و اندازه‌گیری تسریع

تست‌بنچ دوم دو کار انجام می‌دهد که تست اول انجام نمی‌دهد. اول اینکه همان ماژول را با سه پهنای متفاوت ($N = 4, 8, 16$) می‌سازد تا مطمئن شویم هیچ‌جای طراحی به عدد ۸ گره نخورده است. دوم اینکه تعداد کلاک هر ضرب را از خود مدار اندازه می‌گیرد و بررسی می‌کند که واقعاً برابر $N/2 + 1$ باشد؛ یعنی ادعای «شیفت بیش از یک بیت در هر کلاک» به صورت عددی اثبات می‌شود نه اینکه صرفاً ادعا شود.

```

1 `timescale 1ns/1ps
2
3 // Two things this testbench establishes that the main one does not:

```

```

4 //
5 // 1. The design is genuinely parameterised in N (4, 8 and 16 bit
6 // instances all multiply correctly), so nothing is hard-coded to 8.
7 // 2. The radix-4 iteration count really is N/2 and not N - i.e. the
8 // "shift more than one bit per clock" requirement of the lab is met.
9 // The cycle count is MEASURED from the DUT, not asserted.
10 module tb_booth_param;
11
12     integer pass, fail;
13
14     // ----- N = 4 -----
15     reg          Clk4, RstN4, start4;
16     reg signed [3:0] a4, b4;
17     wire signed [7:0] p4;
18     wire          busy4, done4;
19     booth_multiplier #(N(4)) dut4 (
20         .Clk(Clk4), .RstN(RstN4), .start(start4),
21         .Multiplicand(a4), .Multiplier(b4),
22         .Product(p4), .busy(busy4), .done(done4));
23     initial Clk4 = 0;
24     always #5 Clk4 = ~Clk4;
25
26     // ----- N = 8 -----
27     reg          Clk8, RstN8, start8;
28     reg signed [7:0] a8, b8;
29     wire signed [15:0] p8;
30     wire          busy8, done8;
31     booth_multiplier #(N(8)) dut8 (
32         .Clk(Clk8), .RstN(RstN8), .start(start8),
33         .Multiplicand(a8), .Multiplier(b8),
34         .Product(p8), .busy(busy8), .done(done8));
35     initial Clk8 = 0;
36     always #5 Clk8 = ~Clk8;
37
38     // ----- N = 16 -----
39     reg          Clk16, RstN16, start16;
40     reg signed [15:0] a16, b16;
41     wire signed [31:0] p16;
42     wire          busy16, done16;
43     booth_multiplier #(N(16)) dut16 (
44         .Clk(Clk16), .RstN(RstN16), .start(start16),
45         .Multiplicand(a16), .Multiplier(b16),
46         .Product(p16), .busy(busy16), .done(done16));
47     initial Clk16 = 0;
48     always #5 Clk16 = ~Clk16;
49
50     integer cyc4, cyc8, cyc16;
51     integer i, j;
52
53     // NOTE: a bit part-select such as i[7:0] is UNSIGNED in Verilog, so
54     // `i[7:0] * j[7:0]` silently computes an unsigned product. The reference
55     // must be formed from signed regs for the comparison to mean anything.
56     reg signed [3:0] sa4, sb4;
57     reg signed [7:0] sa8, sb8;
58     reg signed [15:0] sa16, sb16;
59     reg signed [7:0] exp4;
60     reg signed [15:0] exp8;
61     reg signed [31:0] exp16;
62
63     task automatic report;
64         input [255:0] tag;
65         input        ok;
66         begin
67             if (ok) pass = pass + 1;
68             else begin
69                 $display("FAIL [%0s]", tag);
70                 fail = fail + 1;
71             end
72         end
73     endtask
74
75     // --- N=4 : exhaustive over all signed 4-bit pairs ---
76     task automatic run4;
77         begin
78             @(negedge Clk4); RstN4 = 0; start4 = 0; a4 = 0; b4 = 0;
79             @(negedge Clk4); RstN4 = 1;
80             for (i = -8; i <= 7; i = i + 1) begin
81                 for (j = -8; j <= 7; j = j + 1) begin
82                     @(negedge Clk4);
83                     sa4 = i[3:0]; sb4 = j[3:0]; exp4 = sa4 * sb4;
84                     a4 = sa4; b4 = sb4; start4 = 1;
85                     @(negedge Clk4); start4 = 0;
86                     // Count clock edges from the one that sampled `start`
87                     // until `done` is observed high.
88                     cyc4 = 0;
89                     while (!done4) begin
90                         @(posedge Clk4); #1; cyc4 = cyc4 + 1;

```

```

91         end
92         #1;
93         report("N4/product", p4 === exp4);
94     end
95 end
96 // 4-bit radix-4 => 2 iterations => 2+1 = 3 clocks to done
97 report("N4/cycles", cyc4 == (4/2) + 1);
98 $display(" N=4 : %0d clocks per multiply (radix-2 would need %0d)",
99         cyc4, 4 + 1);
100 end
101 endtask
102
103 // --- N=8 : sampled pairs incl. the most-negative value ---
104 task automatic run8;
105     begin
106         @(negedge Clk8); RstN8 = 0; start8 = 0; a8 = 0; b8 = 0;
107         @(negedge Clk8); RstN8 = 1;
108         for (i = -128; i <= 127; i = i + 17) begin
109             for (j = -128; j <= 127; j = j + 13) begin
110                 @(negedge Clk8);
111                 sa8 = i[7:0]; sb8 = j[7:0]; exp8 = sa8 * sb8;
112                 a8 = sa8; b8 = sb8; start8 = 1;
113                 @(negedge Clk8); start8 = 0;
114                 cyc8 = 0;
115                 while (!done8) begin
116                     @(posedge Clk8); #1; cyc8 = cyc8 + 1;
117                 end
118                 #1;
119                 report("N8/product", p8 === exp8);
120             end
121         end
122         report("N8/cycles", cyc8 == (8/2) + 1);
123         $display(" N=8 : %0d clocks per multiply (radix-2 would need %0d)",
124             cyc8, 8 + 1);
125     end
126 endtask
127
128 // --- N=16 : sampled pairs, incl. -32768 ---
129 task automatic run16;
130     begin
131         @(negedge Clk16); RstN16 = 0; start16 = 0; a16 = 0; b16 = 0;
132         @(negedge Clk16); RstN16 = 1;
133         for (i = -32768; i <= 32767; i = i + 4447) begin
134             for (j = -32768; j <= 32767; j = j + 5119) begin
135                 @(negedge Clk16);
136                 sa16 = i[15:0]; sb16 = j[15:0]; exp16 = sa16 * sb16;
137                 a16 = sa16; b16 = sb16; start16 = 1;
138                 @(negedge Clk16); start16 = 0;
139                 cyc16 = 0;
140                 while (!done16) begin
141                     @(posedge Clk16); #1; cyc16 = cyc16 + 1;
142                 end
143                 #1;
144                 report("N16/product", p16 === exp16);
145             end
146         end
147         report("N16/cycles", cyc16 == (16/2) + 1);
148         $display(" N=16 : %0d clocks per multiply (radix-2 would need %0d)",
149             cyc16, 16 + 1);
150     end
151 endtask
152
153 initial begin
154     pass = 0; fail = 0;
155     RstN4 = 1; RstN8 = 1; RstN16 = 1;
156     start4 = 0; start8 = 0; start16 = 0;
157     $display("measured iteration counts:");
158     run4;
159     run8;
160     run16;
161     $display("tb_booth_param: Passed: %0d, Failed: %0d", pass, fail);
162     if (fail != 0) $fatal(1);
163     $finish;
164 end
165 endmodule

```

نتایج تست

همه‌ی شبیه‌سازی‌ها با Icarus Verilog 12.0:

نتیجه	پوشش	تست
تمام ۶۵۵۳۶ جفت علامت‌دار ۸ بیتی + حالت‌های مرزی + ریست میانی ۶۵۵۶۲/۶۵۵۶۲	tb_booth_multiplier	
سه پهنای ۴, ۸, ۱۶ و اندازه‌گیری تعداد کلاک ۷۷۴/۷۷۴	tb_booth_param	

خروجی واقعی run_tests.sh:

```
iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_booth_multiplier ===
cycles per multiply: min=5 max=5 (N=8, radix-4 => 4 iterations)
tb_booth_multiplier: Passed: 65562, Failed: 0
PASS: tb_booth_multiplier

=== tb_booth_param ===
measured iteration counts:
  N=4 : 3 clocks per multiply (radix-2 would need 5)
  N=8 : 5 clocks per multiply (radix-2 would need 9)
  N=16 : 9 clocks per multiply (radix-2 would need 17)
tb_booth_param: Passed: 774, Failed: 0
PASS: tb_booth_param

summary: 2/2 passed, 0 failed
```

جمع‌بندی

در این آزمایش دیدیم که تفکیک مسیر داده از واحد کنترل چه فایده‌ای دارد. مسیر داده فقط می‌داند چگونه یک تکرار بوث را انجام دهد و واحد کنترل فقط می‌داند چند بار و چه زمانی این کار باید تکرار شود. نتیجه این است که هیچ‌کدام به جزئیات دیگری وابسته نیستند؛ مثلاً تغییر N از ۸ به ۱۶ فقط تعداد تکرارها را عوض می‌کند و یک خط از واحد کنترل هم تغییر نمی‌کند.

درباره‌ی شرط تسریع، استفاده از بوث پایه-۴ باعث شد تعداد کلاک‌ها از $N + 1$ به $N/2 + 1$ کاهش پیدا کند، یعنی حدود دو برابر سریع‌تر. البته این تسریع رایگان نیست: در ازای آن باید یک جمع‌کننده‌ی پهن‌تر و منطق انتخاب پنج‌حالت برای ضرب‌ها پرداخت شود، که مصالحه‌ی همیشگی سرعت در برابر سطح است.

در نهایت، مهم‌ترین درس عملی این آزمایش مسئله‌ی پهنای انباشتگر بود. حالت $M = -2^{N-1}$ در نگاه اول یک حالت مرزی بی‌اهمیت به نظر می‌رسد، اما دقیقاً همان‌جایی است که $2M$ - سرریز می‌کند و طراحی بی‌سروصدا جواب اشتباه می‌دهد. اینکه تست به‌صورت exhaustive نوشته شده بود باعث شد این اشکال بلافاصله دیده شود.